

Trans-Workflow 项目介绍白皮书

极简 AI 工作流引擎——单文件部署 • 可视化编排 • 节点级回溯 • 纯私有运行

版本：1.0.0 | 日期：2026-06-12 | 作者：Liu Wenjin | 许可：MIT

一、项目概述

1.1 项目名称

Trans-Workflow（极简 AI 工作流）

1.2 一句话介绍

一个纯 HTML 文件承载的轻量级 AI Agent 工作流引擎，无需服务器、无需数据库、无需 Docker，通过可视化编排将多个 AI Agent 串联为高效协同的生产流水线，支持过程透明的人机协同与节点级精准回溯。

1.3 项目定位

Trans-Workflow 是一款面向「意图驱动编程（Intent-Driven Programming）」与「Vibe Coding」开发者的开源 Agent 协同工具。它不追求成为包揽一切的“全能巨兽”，而是专注于做 **最轻量、最透明、最易落地的 Agent 工作流引擎**。

项目基于 **TransWeb 在线 Agent 底座** 构建，每个 Agent 均可通过自然语言以 Vibe 方式直接创建和维护。所有编排逻辑均在本地浏览器运行，不依赖任何后端服务。

1.4 开源地址

- GitHub: <https://github.com/liuwenjin/trans-workflow>
- 在线体验: <https://transweb.cn/workflow>

二、核心功能与特性

2.1 多 Agent 同步串联

支持多个 AI Agent 节点流水线式无缝交接。前置 Agent 的输出 (DataId / InputItem) 自动作为下一个 Agent 的初始输入，无需手动复制粘贴，构建完整的生产链路。

选题 Agent → 大纲 Agent → 内容 Agent → 润色 Agent

2.2 执行过程透明与人工干预（Human-in-the-Loop）

每个 Agent 节点内嵌于高隔离性的 Iframe 容器中，执行状态、网络日志、URL 变更、数据载荷全程可见。开发者可随时暂停流程、手动修改中间结果，再让后续 Agent 基于修改后的数据继续运行，真正实现 AI + Human 的无缝协同。

2.3 节点级回溯与重触发

支持在工作流的任意中间节点一键“重新触发”或“状态重置”，如果第 4 步效果不满意，无需从头重跑前 3 步，直接定位到该节点调整后局部重跑，极大降低 Token 损耗和调试成本。

2.4 动态自由编排

允许在界面上动态新增、删除、调换 Agent 节点，随时重新生成步骤流。编排完成后可导出新版工作流 HTML 文件，方便分享或归档。

2.5 自动持久化与数据主权

工作流执行过程数据及最终产出自动存储在用户个人的 TransWeb Agent 底座后台数据库中。所有执行痕迹、节点状态、最终产物均可随时登录查看、编辑或导出，数据所有权完全归属用户。

2.6 颠覆性的单文件部署

整个系统被极致浓缩进一个 HTML 文件中：

一个 HTML 文件 = 编辑器 + 工作流引擎 + 运行环境

无需 Docker、MySQL、Redis 或任何后端运行时。将其上传至任意 Web 服务器（甚至直接双击本地打开），即可拥有一套完整的 Agent 协同系统。

三、技术架构

3.1 技术栈总览

层次	技术选型	说明
运行容器	纯 HTML + JavaScript	单文件自包含，零依赖部署
UI 框架	Vue.js 3.x + Element plus	核心业务逻辑组件
样式方案	Tailwind CSS	原子化 CSS 框架，响应式布局
构建工具	Node.js（原生 API）	零 npm 依赖的 DataURL 编译脚本
Agent 底座	TransWeb 平台	在线 Agent 创建、执行与存储
跨域通信	HTML5 postMessage	Iframe 跨域状态监听与数据传递
分析追踪	Google Analytics	用户行为埋点

3.2 架构设计理念

Trans-Workflow 的架构遵循“**独立运行与记忆，底座负责时序**”原则：

- 工作流层**：仅负责编排配置和执行状态监听，运行于本地浏览器
- Agent 层**：每个 Agent 运行于 TransWeb 底座提供的沙盒 Iframe 中，拥有独立的记忆、编辑修改、结果版本管理以及文档导出功能
- 数据传递层**：底座根据工作流参数，自动将当前 Agent 的执行结果通过 DataId 引用机制传递给下一个 Agent 节点

3.3 项目目录结构

workflow/	
├── index.html	# 主体运行容器（单文件全自包含）
├── code.vue	# 开发态核心业务逻辑组件（Vue 单文件组件）
├── config.json	# Agent 集群配置文件
├── build.js	# 自动化编译脚本（零 npm 依赖）
├── package.json	# 项目元信息（仅含 build 脚本）
├── readme.md	# 项目说明文档
├── home.html	# 产品落地页 / 营销页面
├── home_bak.html	# 落地页备份
├── docs/	# 文档站点（基于 Docsify）
│ ├── README.md	# 文档首页 / 项目简介
│ ├── _sidebar.md	# 文档侧边栏导航
│ ├── index.html	# Docsify 渲染入口
│ ├── start.md	# 快速开始指南
│ ├── concepts.md	# 核心概念详解
│ ├── guide.md	# 工作流构建指南
│ ├── practices.md	# 最佳实践
│ ├── agent.md	# Agent 节点配置说明
│ ├── nodes.md	# 可用 Agent 案例
│ ├── cases.md	# 案例中心
│ ├── forSkill.md	# 封装工作流 Skill
│ ├── advance.md	# 二次开发进阶
│ ├── deploy.md	# 部署指南
│ ├── faq.md	# 常见问题
└── images/	# 文档配图（24 张截图）

四、核心模块详解

4.1 index.html —— 主体运行容器

index.html 是全自包含的单文件 HTML 应用，**无需任何 Web 服务器**，直接双击即可在浏览器中独立运行完整工作流。其核心机制：

- 内嵌 app.task.data：从 config.json 注入的 Agent 集群配置数据
- 内嵌 app.task.template：以 Base64 DataURL 形式全量嵌入的 code.vue 编译产物
- 通过 TransWeb 底座的 webCpu.updateView() API 动态渲染 Vue 组件

4.2 code.vue —— 核心业务逻辑组件

code.vue 是开发态的核心组件，包含以下关键模块：

模块	功能
UI 视图层	工作流标题编辑、Agent 选择器、步骤条、Iframe 容器、工具栏按钮
多步骤状态机	stepData 数组管理各节点运行时状态，activeStepIndex 跟踪当前执行步骤
Iframe 通信监听	双域兼容的生命周期钩子，实时捕获 Agent 所在 Iframe 的 URL 变动
重置事件处理	支持单节点重触发、全局工作流重置、状态清空
工作流导出	将当前配置序列化为新 HTML 文件下载，支持运行态/快照态两种模式

核心数据结构：

// apps: 可选 Agent 资产元数据池
// accountData.workflow: 当前编排的线性工作流节点 ID 队列
// stepData: 运行时步骤实例状态数组
{
appId: "...", // 当前步骤绑定的 Agent ID
currentNodeId: "...", // 该步骤的初始加载 URL
resultUrl: "...", // 当前步骤完成后的产出 URL
}

4.3 config.json —— Agent 集群配置

集中管理 Agent 集群的元数据及默认工作流的初始化执行顺序：

{
"title": "行业认知工作流",
"apps": [
{ "value": "agent_id", "label": "Agent 名称" }
],
"accountData": {
"appId": "起始 Agent ID",
"workflow": ["后续 Agent ID 列表"]
}
}

4.4 build.js —— 自动化编译脚本

基于 Node.js 原生 API 的构建工具，**零 npm 依赖**，一键完成三步编译：

- 数据注入**：将 config.json 的 JSON 数据无缝注入到 index.html 的 app.task.data 中
- 源码转 DataURL**：将 code.vue 的完整源码转换为 Base64 格式的 DataURL
- 容器代码替换**：精准全量替换 index.html 中的 app.task.template

4.5 DataId 引用机制

Trans-Workflow 创新性地引入了 **DataId 引用机制**，解决了传统工作流中全文复制粘贴导致的传输效率低下、Token 成本暴增等问题：

传统模式：Node A（万字长文）—— 全文网络传输 ——> Node B（高延迟/高成本）
DataId 模式：Node A ——> 写入本地/内存 ——> 返回 DataId ——> Node B 仅引用 ID（毫秒级响应）

核心优势：极致效能、精准回溯、长文本友好。

4.6 跨域 Iframe 状态监听器

code.vue 内建了一套高兼容性的生命周期钩子：

- 同域环境**：直接监听 iframe.contentWindow.location.href
- 跨域环境**：安全监听跨域消息总线 window.addEventListener('message', ...)，当收到 IFRAME_URL_CHANGE 事件时自动更新工作流状态

五、使用场景

5.1 AI 内容全自动/半自动流水线

选题分析 → 大纲生成 → 文章创作 → 润色优化

适用于公众号写作、自媒体矩阵、数字化营销团队的内容标准化生产。

5.2 智能化课件/教学流程设计

需求分析 → 课程大纲 → 知识点拆解 → 页面规划 → 讲义生成

适用于教育机构、企业培训部门的课程内容快速生成。

5.3 意图驱动的 Agent 应用开发

用户输入 → 意图识别 → 任务规划 → 模块执行 → 结果输出

适用于 AI 应用开发者快速验证多 Agent 协作流程并交付 MVP。

5.4 企业轻量知识库工作流

知识检索 → 内容归纳 → 答案生成 → 校验审核

适用于企业内部私有化 AI 工作流、智能办公助手、自动化文档生成。

六、部署与运行说明

6.1 在线体验（零部署）

直接访问 <https://transweb.cn/workflow> 即可在线使用，在线版本为无状态运行模式，完成配置后需手动导出 HTML 文件保存。

6.2 本地运行（单文件部署）

- 访问 <https://transweb.cn/workflow/index.html> 下载 index.html
- 直接双击打开该 HTML 文件，或在浏览器中打开
- 无需安装任何软件、无需配置数据库、无需启动服务器

6.3 源码构建与二次开发

环境要求：Node.js >= 14.0.0

构建步骤：

1. 克隆仓库
git clone https://github.com/liuwenjin/trans-workflow.git
cd trans-workflow/static/workflow
2. 修改 config.json 中的 Agent ID 为你的真实 Agent ID
3. 一键编译
node build.js
4. 运行
双击 index.html 或使用任意静态文件服务器
open index.html

6.4 创建 Agent 节点

- 访问 [agent.transweb.cn](https://transweb.cn) 注册/登录
- 使用自然语言描述需求，创建 AI Agent
- 记录每个 Agent 的唯一 ID（可在地址栏中找到）
- 将 Agent ID 填入 config.json 中即可

七、与主流平台对比

维度	Trans-Workflow	Dify	Langflow	n8n
工作流干预度	中间结果完全可见、可手动编辑	运行中难以动态拦截修改	侧重数据流，干预成本高	偏向自动化，交互干预弱
部署成本	单 HTML 文件，零门槛、零依赖	依赖 Docker 及多套数据库	依赖 Python 环境及包管理	依赖 Node.js 运行时与后端
私有化体验	无缝开箱即用，纯前端解耦	配置与维护成本较高	生产环境配置较重	涉及证书与权限，配置繁琐
节点/算力限制	完全无限制	云端版本存在免费额度限制	受限于本地环境与配置	触发次数或活跃节点受限
商业闭源风险	永久免费，源码完全自主掌控	商业版与开源版功能分化	企业级高级功能收费	调整了开源协议，商业受限

八、总结

Trans-Workflow 彻底打破了 Agent 编排的繁琐壁垒。用一个 HTML 文件，完美闭环了 Agent 协同网络的可视化编排、透明执行、精细调试与极简部署。

在开发工具越来越重、概念越来越复杂的今天，Trans-Workflow 追求的是**克制与纯粹**——让工作流回归本质：Agent 的协作与结果控制。通过抹去语法与环境的噪音，让开发者将全部精力聚焦在“意图表达”与“内容打磨”上。

核心公式：

Workflow = Agent 协作链 + 过程完全透明 + 全链路人工干预（Human-in-the-Loop）

